



Name: \_\_\_\_\_ Cohort/Batch: \_\_\_\_\_ Date: \_\_\_\_\_ Score: \_\_\_\_\_

## HASHICORP VAULT PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 conceptual Q&A Security

TOTAL: 10 QUESTIONS

**Q1** What is HashiCorp Vault and what security threat does it address? Simulation

---

---

**Q6** How do you audit and monitor HashiCorp Vault events? Observability

---

---

**Q2** What is the core mechanism HashiCorp Vault uses to provide security? Core

---

---

**Q7** How does HashiCorp Vault handle key or credential rotation? Policy

---

---

**Q3** How do you configure HashiCorp Vault for a production application? Configuration

---

---

**Q8** How does HashiCorp Vault help meet compliance requirements? Compliance

---

---

**Q4** What are the most common HashiCorp Vault misconfigurations to avoid? Configuration

---

---

**Q9** How do you test your HashiCorp Vault configuration for weaknesses? Testing

---

---

**Q5** How does HashiCorp Vault integrate with identity providers? IdP

---

---

**Q10** What are the performance implications of HashiCorp Vault and how do you minimize them? Performance

---

---



ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

A1 HashiCorp Vault is a security tool, protocol,

Mitigation

HashiCorp Vault is a security tool, protocol, or service that mitigates a specific class of threats: authentication bypass, data exfiltration, network intrusion, or credential theft. Understanding the threat model is prerequisite to correct configuration.

A6 Enable HashiCorp Vault's audit log and stream

Observability

Enable HashiCorp Vault's audit log and stream it to a SIEM (Splunk, Elastic SIEM). Define alert rules for high-severity events: repeated failed logins, privilege escalation, and unusual access patterns. Review logs regularly.

A2 HashiCorp Vault uses one or more security

Primitives

HashiCorp Vault uses one or more security primitives: asymmetric cryptography, symmetric encryption, certificate chains, role-based access control, or anomaly detection. These primitives are combined to provide the claimed security properties.

A7 Automate rotation using a secrets manager that

Automation

Automate rotation using a secrets manager that generates new credentials and updates consumers transparently. Test rotation in staging. Have a break-glass procedure for emergency rotation when a credential is compromised.

A3 Production configuration requires strong cipher suites,

Hardening

Production configuration requires strong cipher suites, certificate rotation, revocation checking, and minimal permission grants. Use a well-reviewed configuration reference (CIS Benchmark, OWASP) rather than default settings.

A8 HashiCorp Vault generates audit trails, enforces access,

Compliance

HashiCorp Vault generates audit trails, enforces access policies, and provides encryption that satisfy controls required by SOC 2, PCI-DSS, HIPAA, and GDPR. Map HashiCorp Vault's capabilities to the specific framework controls in your compliance program.

A4 Common mistakes include using outdated algorithms (MD5, SHA1, RC4),

MD5 has

Common mistakes include using outdated algorithms (MD5, SHA1, RC4), leaving default credentials, ignoring certificate expiry, granting excessive permissions, and not testing the config before going live in production.

A9 Run an automated scanner (SSL Labs for TLS, ScoutSuite for cloud IAM, OWASP ZAP for web app auth) against your HashiCorp Vault config. Conduct penetration tests annually and after significant config changes.

Assessment

A5 HashiCorp Vault delegates identity verification to an

Federation

HashiCorp Vault delegates identity verification to an IdP via SAML, OIDC, or LDAP. Users authenticate once (SSO) and receive a token that HashiCorp Vault validates. This centralizes user management and avoids duplicating auth logic across services.

A10 Cryptographic operations, token validation, and authorization

Authorization

Cryptographic operations, token validation, and authorization checks add latency. Mitigate with session caching, hardware-accelerated TLS (AES-NI), connection reuse, and async authorization where possible.